

Database Source Code

Connection Config, Schema Builder & SQL Schema

Table of Contents

Database

- db/config.js
- db/schema.js
- db/schema.sql

```

1  import "dotenv/config";
2  import path from "path";
3  import { fileURLToPath } from "url";
4  import mysql from "mysql2/promise";
5
6  const __filename = fileURLToPath(import.meta.url);
7  const __dirname = path.dirname(__filename);
8
9  // Database connection pool
10 export const db = mysql.createPool({
11   host: process.env.DB_HOST || "localhost",
12   user: process.env.DB_USER || "root",
13   // Double-check your .env: usually this variable is DB_PASSWORD, not DB_PASS
14   password: process.env.DB_PASS || "",
15   database: process.env.DB_NAME || "evangadi_forum",
16   waitForConnections: true,
17   connectionLimit: 10,
18   queueLimit: 0
19 });
20
21
22 const ensureParams = (params) => {
23   if (params === undefined || params === null) {
24     throw new Error("SQL parameters are required");
25   }
26   const isArray = Array.isArray(params);
27   const isObject = !isArray && typeof params === "object";
28   if (!isArray && !isObject) {
29     throw new Error("SQL parameters must be an array or object");
30   }
31 };
32
33 export const safeExecute = async (sql, params) => {
34   if (typeof sql !== "string" || sql.trim().length === 0) {
35     throw new Error("SQL query must be a non-empty string");
36   }
37   ensureParams(params);
38   const [result] = await db.execute(sql, params);
39   return result;
40 };
41
42 // import dotenv from "dotenv";
43 // dotenv.config();
44 // import mysql from "mysql2/promise";
45
46 // // Database connection pool
47 // export const db = mysql.createPool({
48 //   host: process.env.DB_HOST || "localhost",
49 //   port: process.env.DB_PORT || 8889,
50 //   user: process.env.DB_USER || "root",
51 //   password: process.env.DB_PASS || "",
52 //   database: process.env.DB_NAME || "forumPage",
53 // });
54
55 // const ensureParams = (params) => {
56 //   if (params === undefined || params === null) {
57 //     throw new Error("SQL parameters are required");
58 //   }
59 //   const isArray = Array.isArray(params);
60 //   const isObject = !isArray && typeof params === "object";
61 //   if (!isArray && !isObject) {
62 //     throw new Error("SQL parameters must be an array or object");
63 //   }
64 // };
65
66 // export const safeExecute = async (sql, params) => {
67 //   if (typeof sql !== "string" || sql.trim().length === 0) {
68 //     throw new Error("SQL query must be a non-empty string");
69 //   }
70 //   ensureParams(params);
71 //   const [result] = await db.execute(sql, params);
72 //   return result;
73 // };

```

```

1  import { db } from './config.js';
2
3  const createTables = async () => {
4    try {
5      await db.execute(`SET FOREIGN_KEY_CHECKS = 0`);
6
7      // USERS
8      await db.execute(`
9        CREATE TABLE IF NOT EXISTS users (
10          user_id INT AUTO_INCREMENT PRIMARY KEY,
11          first_name VARCHAR(50) NOT NULL,
12          last_name VARCHAR(50) NOT NULL,
13          email VARCHAR(320) NOT NULL UNIQUE,
14          password_hash VARCHAR(255) NOT NULL,
15          created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
16          updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
17          CHECK (email = LOWER(email)),
18          INDEX idx_users_email (email)
19        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
20      `);
21
22      // QUESTIONS
23      await db.execute(`
24        CREATE TABLE IF NOT EXISTS questions (
25          question_id INT AUTO_INCREMENT PRIMARY KEY,
26          question_hash CHAR(16) NOT NULL UNIQUE,
27          user_id INT NOT NULL,
28          title VARCHAR(255) NOT NULL,
29          content TEXT NOT NULL,
30          created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
31          updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
32
33          CHECK (CHAR_LENGTH(title) >= 5),
34          CHECK (CHAR_LENGTH(content) >= 10),
35
36          FOREIGN KEY (user_id)
37            REFERENCES users(user_id)
38            ON DELETE CASCADE,
39
40          INDEX idx_questions_user_id (user_id),
41          INDEX idx_questions_created_at (created_at),
42          FULLTEXT KEY ft_questions_search (title, content)
43        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
44      `);
45
46      // QUESTION VECTORS
47      await db.execute(`
48        CREATE TABLE IF NOT EXISTS question_vectors (
49          vector_id BIGINT AUTO_INCREMENT PRIMARY KEY,
50          question_id INT NOT NULL,
51          source_text TEXT NOT NULL,
52          embedding JSON NOT NULL,
53          status VARCHAR(20) DEFAULT 'ready',
54          created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
55          updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
56
57          FOREIGN KEY (question_id)
58            REFERENCES questions(question_id)
59            ON DELETE CASCADE,
60
61          UNIQUE KEY uniq_question_vectors_question_id (question_id)
62        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
63      `);
64
65      // ANSWERS
66      await db.execute(`
67        CREATE TABLE IF NOT EXISTS answers (
68          answer_id INT AUTO_INCREMENT PRIMARY KEY,
69          question_id INT NOT NULL,
70          user_id INT NOT NULL,
71          content TEXT NOT NULL,
72          created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
73          updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
74
75          FOREIGN KEY (question_id)
76            REFERENCES questions(question_id)
77            ON DELETE CASCADE,
78
79          FOREIGN KEY (user_id)
80            REFERENCES users(user_id)
81            ON DELETE CASCADE,
82
83          INDEX idx_answers_question_id (question_id),
84          INDEX idx_answers_user_id (user_id),
85          INDEX idx_answers_created_at (created_at)
86        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
87      `);
88
89      // DOCUMENTS
90      await db.execute(`
91        CREATE TABLE IF NOT EXISTS documents (

```

```

92     document_id INT AUTO_INCREMENT PRIMARY KEY,
93     user_id INT NOT NULL,
94     title VARCHAR(512) NOT NULL,
95     mime_type VARCHAR(128) NOT NULL DEFAULT 'application/pdf',
96     storage_path VARCHAR(1024) NOT NULL,
97     byte_size BIGINT NOT NULL DEFAULT 0,
98     status VARCHAR(20) NOT NULL DEFAULT 'pending',
99     error_message TEXT NULL,
100    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
101    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
102
103    FOREIGN KEY (user_id)
104        REFERENCES users(user_id)
105        ON DELETE CASCADE,
106
107    INDEX idx_documents_user_created (user_id, created_at)
108 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
109 `);
110
111 // DOCUMENT CHUNKS
112 await db.execute(`
113     CREATE TABLE IF NOT EXISTS document_chunks (
114         chunk_id BIGINT AUTO_INCREMENT PRIMARY KEY,
115         document_id INT NOT NULL,
116         chunk_index INT NOT NULL,
117         content TEXT NOT NULL,
118         page_start INT NULL,
119         page_end INT NULL,
120         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
121
122         FOREIGN KEY (document_id)
123             REFERENCES documents(document_id)
124             ON DELETE CASCADE,
125
126         UNIQUE KEY uniq_document_chunks_doc_index (document_id, chunk_index),
127         INDEX idx_document_chunks_document_id (document_id)
128     ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
129 `);
130
131 // DOCUMENT CHUNK VECTORS
132 await db.execute(`
133     CREATE TABLE IF NOT EXISTS document_chunk_vectors (
134         chunk_vector_id BIGINT AUTO_INCREMENT PRIMARY KEY,
135         chunk_id BIGINT NOT NULL,
136         source_text TEXT NOT NULL,
137         embedding JSON NOT NULL,
138         status VARCHAR(20) NOT NULL DEFAULT 'ready',
139         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
140         updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
141
142         FOREIGN KEY (chunk_id)
143             REFERENCES document_chunks(chunk_id)
144             ON DELETE CASCADE,
145
146         UNIQUE KEY uniq_chunk_vectors_chunk_id (chunk_id)
147     ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
148 `);
149
150 await db.execute(`SET FOREIGN_KEY_CHECKS = 1`);
151
152 console.log("☑ All Evangadi Forum tables created successfully.");
153 process.exit(0);
154 } catch (error) {
155     console.error("❌ Error creating tables:", error);
156     process.exit(1);
157 }
158 };
159
160 createTables();

```

```

1  -- Database Schema for Evangadi Forum
2  -- Platform: MySQL
3
4  SET FOREIGN_KEY_CHECKS = 0;
5
6  -----
7  -- 1. Users Table
8  -- Stores user account information.
9  -----
10 DROP TABLE IF EXISTS `users`;
11 CREATE TABLE `users` (
12     `user_id` INT AUTO_INCREMENT PRIMARY KEY,
13     `first_name` VARCHAR(50) NOT NULL,
14     `last_name` VARCHAR(50) NOT NULL,
15     `email` VARCHAR(320) NOT NULL UNIQUE,
16     `password_hash` VARCHAR(255) NOT NULL,
17     `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
18     `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
19     CHECK (`email` = LOWER(`email`)),
20
21     INDEX `idx_users_email` (`email`)
22 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
23
24 -----
25 -- 2. Questions Table
26 -- Stores the main questions posted by users.
27 -- Supports full-text search on title and content for exact match search.
28 -----
29 DROP TABLE IF EXISTS `questions`;
30 CREATE TABLE `questions` (
31     `question_id` INT AUTO_INCREMENT PRIMARY KEY,
32     `question_hash` CHAR(16) NOT NULL UNIQUE, -- Used for /question/:hash routing
33     `user_id` INT NOT NULL,
34     `title` VARCHAR(255) NOT NULL,
35     `content` TEXT NOT NULL, -- Detailed content including code sections
36     `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
37     `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
38     CHECK (CHAR_LENGTH(`title`) >= 5),
39     CHECK (CHAR_LENGTH(`content`) >= 10),
40
41     FOREIGN KEY (`user_id`) REFERENCES `users`(`user_id`) ON DELETE CASCADE,
42
43     INDEX `idx_questions_user_id` (`user_id`),
44     INDEX `idx_questions_created_at` (`created_at`),
45
46     -- Full-text search index for exact match search mode
47     FULLTEXT KEY `ft_questions_search` (`title`, `content`)
48 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
49
50 -----
51 -- 3. Question Vectors Table
52 -- Stores embeddings for the AI Semantic Search feature (Gemini default model).
53 -----
54 DROP TABLE IF EXISTS `question_vectors`;
55 CREATE TABLE `question_vectors` (
56     `vector_id` BIGINT AUTO_INCREMENT PRIMARY KEY,
57     `question_id` INT NOT NULL,
58     `source_text` TEXT NOT NULL, -- Text used to generate the embedding
59     `embedding` JSON NOT NULL, -- Gemini embedding vector
60     `status` VARCHAR(20) DEFAULT 'ready', -- e.g., 'ready', 'pending', 'failed'
61     `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
62     `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
63     FOREIGN KEY (`question_id`) REFERENCES `questions`(`question_id`) ON DELETE CASCADE,
64     UNIQUE KEY `uniq_question_vectors_question_id` (`question_id`)
65 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
66
67 -----
68 -- 4. Answers Table
69 -- Stores answers to questions.
70 -----
71 DROP TABLE IF EXISTS `answers`;
72 CREATE TABLE `answers` (
73     `answer_id` INT AUTO_INCREMENT PRIMARY KEY,
74     `question_id` INT NOT NULL,
75     `user_id` INT NOT NULL,
76     `content` TEXT NOT NULL, -- Content including code sections
77     `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
78     `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
79
80     FOREIGN KEY (`question_id`) REFERENCES `questions`(`question_id`) ON DELETE CASCADE,
81     FOREIGN KEY (`user_id`) REFERENCES `users`(`user_id`) ON DELETE CASCADE,
82
83     INDEX `idx_answers_question_id` (`question_id`),
84     INDEX `idx_answers_user_id` (`user_id`),
85     INDEX `idx_answers_created_at` (`created_at`)
86 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
87
88 -----
89 -- 5. RAG: user-owned PDF documents, text chunks, and chunk embeddings
90 -----
91 DROP TABLE IF EXISTS `documents`;

```

```
92 CREATE TABLE `documents` (  
93   `document_id` INT AUTO_INCREMENT PRIMARY KEY,  
94   `user_id` INT NOT NULL,  
95   `title` VARCHAR(512) NOT NULL,  
96   `mime_type` VARCHAR(128) NOT NULL DEFAULT 'application/pdf',  
97   `storage_path` VARCHAR(1024) NOT NULL,  
98   `byte_size` BIGINT NOT NULL DEFAULT 0,  
99   `status` VARCHAR(20) NOT NULL DEFAULT 'pending',  
100  `error_message` TEXT NULL,  
101  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
102  `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
103  FOREIGN KEY (`user_id`) REFERENCES `users` (`user_id`) ON DELETE CASCADE,  
104  INDEX `idx_documents_user_created` (`user_id`, `created_at`)  
105 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;  
106  
107 DROP TABLE IF EXISTS `document_chunks`;  
108 CREATE TABLE `document_chunks` (  
109   `chunk_id` BIGINT AUTO_INCREMENT PRIMARY KEY,  
110   `document_id` INT NOT NULL,  
111   `chunk_index` INT NOT NULL,  
112   `content` TEXT NOT NULL,  
113   `page_start` INT NULL,  
114   `page_end` INT NULL,  
115   `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
116   FOREIGN KEY (`document_id`) REFERENCES `documents` (`document_id`) ON DELETE CASCADE,  
117   UNIQUE KEY `uniq_document_chunks_doc_index` (`document_id`, `chunk_index`),  
118   INDEX `idx_document_chunks_document_id` (`document_id`)  
119 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;  
120  
121 DROP TABLE IF EXISTS `document_chunk_vectors`;  
122 CREATE TABLE `document_chunk_vectors` (  
123   `chunk_vector_id` BIGINT AUTO_INCREMENT PRIMARY KEY,  
124   `chunk_id` BIGINT NOT NULL,  
125   `source_text` TEXT NOT NULL,  
126   `embedding` JSON NOT NULL,  
127   `status` VARCHAR(20) NOT NULL DEFAULT 'ready',  
128   `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
129   `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
130   FOREIGN KEY (`chunk_id`) REFERENCES `document_chunks` (`chunk_id`) ON DELETE CASCADE,  
131   UNIQUE KEY `uniq_chunk_vectors_chunk_id` (`chunk_id`)  
132 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;  
133  
134 SET FOREIGN_KEY_CHECKS = 1;
```